

Space Invaders: 1. Sprint-aren MVC

Txostena

Asier, Ibai Go, Lander, Oier

2026.eko martxoaren 15

1 Deskribapena

Txosten honetan gure Space Invaders proiektuaren 1. sprint-eko arkitektura azalduko dugu. Proiektuan Model-View-Controller (MVC) diseinu-patroia inplementatu dugu logika eta interfaze grafikoa ondo banatzeko. Irakasleak klasean aipatu bezala, kontroladorearen ardura teklatuaren sarrerak kudeatzeari baino ez da, eta gainerako logika, jokoaren begizta (Game Loop) barne, ereduari dagokio.

Gure proiektua honela banatu dugu:

- **Eredua (Model):** Jokoaren muina da. Hemen datuak, egoera eta jokoaren arau guztiak gordetzen dira. Bi klase nagusitan banatu dugu logika, eta bietan Singleton patroia (EMA) inplementatu dugu, joko osoan zehar instantzia bakarra izateko:
 - **MatrizeEredua:** Jokoaren taula fisikoa eta elementuen posizioak kudeatzen ditu. 100×60 tamainako `Gelaxka` matrize bat dauka. `matrizeaSortu()` exekutatzean, gelaxka bakoitzari `Edukia` enum-eko balio bat ematen dio (`Hutsa`, `Horma`, etab.). Bertan sortzen da `JokalariOntzi`-a (hasierako koordinatuetan), baita `Etsaiak` ere (ausaz 4 eta 8 etsai artean sortzen dira matrizearen 5. lerroan). Errendimendua optimizatzeko, `gelaxkaCambiadasList` izeneko zerrenda bat erabiltzen du: matrize osoko 6000 gelaxkak eguneratu beharrean, aldatu diren gelaxkak soilik gordetzen ditu bista azkarrago eguneratzeko. Era berean, klase honek tiroak posizio bat gora mugitzen ditu (`jokoZikloaEguneratu()`) eta etsaien ausazko mugimenduak kontrolatzen ditu (`etsaiakMugitu()`).

- **JokoKudeaketa:** Jokoaren motorra edo "Game Loop-a da. `Timer` bat dauka 50 milisegunduro `jokoZikloa()` metodoa etengabe exekutatzen duena. Kontroladoreak pasatzen dizkion tekla `teclasPresionadas` izeneko `HashSet` batean irakurtzen ditu, ontziaren mugimendu jarraitua eta tiroak ahalbidetuz. Oso garrantzitsua da `tickKontagailua` aldagaiak egiten duena: jokia 50ms-ro berritzen bada ere, etsaiak 4 tick-etik behin (hau da, 200 milisegunduro) mugitzen dira, abiadura egokia mantentzeko. Begiztaren iterazio bakoitzean `egiaztatuAmaiera()` exekutatzen du baldintzak aztertzeko: etsairik geratzen ez bada irabazi egiten dugu; aldez, etsai bat ontziaren alboko gelaxkara iristen bada edo matrizearen behekaldean irteten bada, jokia berrehala amaitzen da galduz.
- **Bista (View):** Erabiltzaileak ikusten duen interfaze grafikoa da. Bista nagusia `LeihoNagusia` da, eta `CardLayout` bat erabiltzen du panelen artean nabigatzeko (Hasiera, Jokoa, Irabazi, Galdu). Jokoaren taula `JokoPanela`-n dago, `GridLayout(60, 100)` egitura erabiliz. Sare honen barruan `GelaxkaBista` objektuak daude (`JLabel`-ak), eta ereduaren arabera beren kolorea aldatzen dute (Berdea ontziarentzat, Gorria etsaiarentzat, Zuria tiroarentzat, etab.).
- **Kontroladorea (Controller):** Gure kasuan, erabiltzailearen ekintzak (teklaturako sarrerak) harrapatzeaz arduratzen da, zehazki `LeihoNagusia`-n kokatutako entzuleen bidez.

2 Kontroladoreek zer egiten dute?

Gure proiektuan kontroladoreak zeregin bakarra dauka: teklaturari adi egotea eta erabiltzailearen aginduak ereduari pasatzea. Gure inplementazioan, arduradua hau `LeihoNagusia` klaseak bere gain hartzen du, Java-ko `KeyListener` interfazea inplementatuz.

Kontroladore gisa, bere funtzionamendua oso sinplea eta zuzena da:

- **Tekla harrapatu:** `keyPressed(KeyEvent e)` eta `keyReleased(KeyEvent e)` metodoen bidez, teklaturan zerbait sakatzen den edo askatzen den detektatzen du. Geziak (gora, behera, ezkerra, eskuina) eta zuriunea (tiroa) bakarrik interesatzen zaizkio.
- **Ereduari informazioa pasa:** Tekla bat detektatzean, ez du jokoko ezer zuzenean aldatzen. Horren ordez, Ereduan dagoen `JokoKudeaketa`-ri abisatzen dio: adibidez, ezkerreko gezia sakatzean, `JokoKudeaketa.getJokoKudeaketa()` exekutatzen du, edo askatzean `teklaAskatu("EZKERRA")`.

Horrela, kontroladorea teklatuaren eta ereduaren arteko zubi hutsa da. Informazio hori `JokoKudeaketa`-ra iristen da zuzenean, eta honek bere begiztaren bitartez jokoaren ekintzak gauzatzen ditu.

3 Observer KD (Klase Diagrama azalpena)

Observer patroia oso garrantzitsua izan da Ereduak Bistari pantaila eguneratu behar duela esateko, elkarren arteko dependentzia zuzenik izan gabe (bistak ereduari buruzko xehetasunak baino ez daki). Gure proiektuan bi mailatan aplikatu dugu Observer:

1. Jokoaren egoera orokorra (Pantaila aldaketak):

- **Observable (Igorlea):** `JokoKudeaketa` (Eredua).
- **Observer (Entzulea):** `LeihoNagusia` (Bista/Kontroladorea).

Jokoa hasieratzen edo amaitzen denean, ereduak abisua bidaltzen du eta leihoak erakusten den panela aldatzen du.

2. Matrizeko Gelaxkak (Jokoaren taula):

- **Observable (Igorlea):** `Gelaxka` (Ereduan).
- **Observer (Entzulea):** `GelaxkaBista` (Bistan).

`LeihoNagusia`-ko `update` metodoan, jokoa hasieratzen denean, matritzeko `Gelaxka` bakoitzari bere posizioko `GelaxkaBista` lotzen diogu (`addObserver` erabiliz). Horrela, logika aldetik gelaxka bat aldatzen bada (adibidez, etsai bat mugitu delako), bere bista grafikoa zuzenean ohartarazten da.

4 Notify/Update, noiz egiten dira?

Observer patroia funtzionamendurako, `notify` eta `update` metodoen erabilera ondo definituta daukagu gure proiektuan.

4.1 Noiz egiten da Notify?

`notify` (gure kodean `notifyObservers()`) ereduaren egoera bat aldatu dela abisatzeko erabiltzen da:

- **Gelaxka.java-n:** Gelaxka baten edukia aldatzen denean, gure `notifikatu()` metodoa exekutatzen da (honek `setChanged()` eta `notifyObservers(edukia.name())` egiten ditu). Dena den, etengabe abisuak ez bidaltzeko, hau `MatrizeEredua`-tik deitzen da soilik joko ziklo bakoitzaren amaieran, `bistaEguneratu()` metodoaren bidez. Honi esker, `gelaxkaCambiadasList`-en gordetako gelaxkek bakarrik egiten dute `notify`, jokia ez moteltzeko.
- **JokoKudeaketa.java-n:** Jokoa hasieratzen denean (matrizea sortzean "MTRX_SORTUTA" bidaliz) edo jokia amaitzean (irabaztean "IRABAZI" edo galtzean "GALDU"), `setChanged()` egin ondoren `notifyObservers(mezua)` deitzen da bista orokorrari zer pantaila erakutsi behar duen esateko.

4.2 Noiz egiten da Update?

`update` metodoa Bistan automatikoki exekutatzen da, Ereduek `notify` egiten duen bakoitzean:

- **GelaxkaBista.java-n:** Gelaxka-k abisua bidaltzen dion bakoitzean, bere `update(Observable o, Object arg)` exekutatzen da. Jasotako objektua `String` bat da (adibidez, "EspazioOntzia", "Etsaia" edo "Tiroa"). Mezua aztertzen du eta, horren arabera, `JLabel`-aren atzeko kolorea aldatzen du. Prozesu hau `SwingUtilities.invokeLater` erabiliz exekutatzen dugu interfazea modu seguruan margotzeko gailu grafikoan.
- **LeihoNagusia.java-n:** `JokoKudeaketa`-k egoera-aldaketa bat notifikatzen duenean exekutatzen da. Abisua "MTRX_SORTUTA" bada, jokoa-ren taulako matrize grafikoa prestatzen du eta `CardLayout`-a erabiliz jokoaren panela pantailaratzen du. Aldiz, "IRABAZI" edo "GALDU" jasotzen badu, amaierako pantaila erakusten du.